

CT-353 Operating Systems

Complex Computing Problem (CCP)

Course Learning Outcome:

CLO 1 (C2): Describe the concepts, structure and design of operating systems.

CLO 2 (C4): Analyze operating system design and its impact on application design and performance.

Complex Problem Solving Attributes:

- **CP-2 Depth of Analysis Required:** Has no obvious solution, and requires conceptual thinking and innovative analysis to formulate suitable abstract models.
- **CP-8 Interdependence:** Is a high-level problem possibly including many component parts or sub-problems.
- **CP-9 Requirements Identification:** Identification of a requirement or the cause of a problem is ill-defined or unknown.

Project Statement:

Design and implement an Operating System project that demonstrates the use of system calls on both Linux and Windows. Students will select, implement, and compare 15 system calls and integrate them into a Fourier Transform computation. The project emphasizes system programming, process and memory management, IPC, and performance evaluation. The project will culminate in a report and presentation documenting all implementations, performance results, and OS-specific observations.

Problem Description:

The project must include:

1. Learn and implement **essential system calls** on Linux and Windows.
2. Compare **system call performance** between the two operating systems.
3. Implement a **Fourier Transform** that uses system-call-based file I/O, memory mapping, and process management.
4. Understand **OS-level mechanisms** like process creation, memory allocation, and IPC.
5. Document findings and performance metrics in a **structured report**.

Assumptions & Constraints:

Assumptions:

- Project work will be done in **groups of 3**.
- Students have access to **Linux and Windows systems**.
- Programming languages: **C/C++ preferred**.

Constraints:

- Implement 15 system calls provided in the list.
- Perform file I/O and memory operations using system calls.

- Parallelize Fourier Transform computation using processes/threads and IPC.
- Use multi-process parallelism to speed up FFT computation.
- Include error handling and logging for all system calls.
- Collect execution time and resource usage for each system call on Linux and Windows.
- Modular and maintainable code.
- Cross-platform compatibility (Linux + Windows).
- Performance metrics documented and reproducible.

Category	Linux	Windows	Purpose
File I/O	open(), read(), write(), close()	CreateFile(), ReadFile(), WriteFile(), CloseHandle()	File operations
Process	fork(), exec(), wait()	CreateProcess(), WaitForSingleObject()	Process creation/control
Memory	mmap(), munmap()	VirtualAlloc(), VirtualFree()	Memory mapping
IPC	pipe(), shmget(), shmat()	CreatePipe(), CreateFileMapping(), MapViewOfFile()	Inter-process communication
Permissions	chmod(), chown(), umask()	SetFileSecurity()	File permissions

Table1: 15 System Calls.

Deliverables:

1. System Call Implementation

- Each group must implement 15 system calls.
- Demonstrate usage with a small program.
- Record performance metrics (execution time, CPU/memory usage).

2. Fourier Transform Integration

- Use system calls for:
- Reading input signal from file.
- Memory-mapped arrays for FFT computation.
- Parallel processing using processes or threads.
- Writing FFT output to file.

3. Performance Comparison

- Compare each system call's performance between Linux and Windows.
- Document differences and observations.

4. Report

- Description of system calls used.

- Fourier Transform implementation details.
- Performance tables and charts.
- Lessons learned and OS-specific behaviors.

5. Presentation

- Maximum 10–12 slides.
- Highlights: system call usage, FFT implementation, performance comparison, conclusions.

Important Note on Plagiarism:

* If any student is found to have copied work from another student, both students will receive 0 marks.

Department of Computer Science & Information Technology

Complex Computing Problem Assessment Rubrics
CT-353 Operating Systems

Student Name: _____

Student Roll No.: _____

Criteria and Scales		
Satisfactory (2)	Average (1)	Unsatisfactory (0)
Criterion 1: Clarity of Objective: To what extent has the student understood the objective? (CP9: Requirement Identification)		
The student has correctly identified the requirements.	The student has correctly identified some requirements.	The student has misunderstood the requirements
Criterion 2: Innovation: To what extent has the student used innovative analysis? (CP2: Depth of Analysis Required)		
The student has used conceptual thinking and innovative analysis to formulate suitable abstract models.	The student has used some conceptual analysis to formulate suitable abstract models.	The student has not performed thorough analysis and the model lacks required concepts.
Criterion 3: Modularity: To what extent has the student been able to identify the interdependent functionalities? (CP8: Interdependence)		
The student has clearly divided the problem with respect to the functionalities and their interactions.	The student has divided the problem with respect to the functionalities and their interactions to a limited extent.	The student has not been able to divide the problem with respect to the functionalities and their interactions.
Criterion 4: Validation: To what extent has the student rationalized the solution? (CP2: Depth of Analysis Required)		
The student has clearly justified that the solution meets the requirements.	The student has partially justified that the solution meets the requirements.	The student has not been able to justify that the solution meets the requirements..
Criterion 5: Planning: To what extent has the student formulated a realistic plan? (CP8: Interdependence)		
The student has formulated a realistic plan considering the risks and constraints.	The student has formulated a partially realistic plan considering the risks and constraints.	The student has formulated an unrealistic plan.

Marks: _____/10

Teacher's Signature: _____